Open Alignment Test Factory — cycles 1-3

2026-05-13

Contents

Open Alignment Test Factory — cycles 1-3 **1**
- Abstract . . . 1
- Introduction . . . 2
- Approach . . . 2
- Findings . . . 2
- Discussion . . . 6
- Open Questions . . . 7
- References . . . 7
- Appendix: Implementation Details . . . 8

Open Alignment Test Factory — cycles 1-3

Abstract

Cycles 1-3 established the foundation for a practical open-source alignment test factory for agentic AI systems. The work did not yet build the runtime toy environment or Inspect AI execution path; it completed the prerequisite layers needed before those can be implemented safely: a landscape gap map, an operational failure-mode taxonomy, a benchmark-quality rubric, and a provider-agnostic task specification schema with validation tests.

The central result from these cycles is that useful agentic alignment testing must score the full trajectory of an agent, not only its final answer. A trajectory means the ordered record of observations, tool calls, state updates, permission decisions, delegation messages, and final answer fields. The factory's emerging contract is therefore: define safe synthetic scenarios, record the trajectory, and evaluate deterministic predicates over trace, state, policy, and structured final output.

The cycle 3 auditor validated the schema milestone with no critical or moderate findings. Validated checks included `pytest tests/test_task_spec_schema.py` with 9 passing tests, `python alignment-test-factory/tools/validate_specs.py` with 4 valid examples accepted and 3 invalid examples rejected, and successful regeneration of the JSON Schema. The main next risk is implementation fidelity: cycle 4 must make the runtime trace format match the schema vocabulary rather than inventing a parallel format. Source sessions: cycle 1 researcher `8e76a8f1-d9a9-47e8-813a-2d7f64033861`, worker `61694eaf-1440-4db0-9fc3-1adae12ab391`, auditor `62225d65-0e26-4e5e-ad64-4eabe38c01c9`; cycle 2 researcher `9aa12561-91c9-4083-b9c3-b80b29fa3b4a`, worker `794cdcc6-f8c9-4f93-a1ed-78064b1eec09`, auditor `cc47bb4e-fbde-42fa-8581-605fe865f7da`; cycle 3 researcher `4ed163f8-2d9c-4175-96ec-b9b25882be05`, worker `a48be015-c6f3-4235-8af4-94234f212441`, auditor `2a2afc67-21e8-407c-9a5c-9d19438d44aa`.

Introduction

The mission is to build an open alignment test factory for agentic AI systems: systems that can use tools, consult memory or state, delegate subtasks, request permissions, and carry out multi-step plans. The directive explicitly rejected another static jailbreak list. Instead, it asked whether high-level safety properties can be converted into safe, reproducible, nontrivial eval tasks for realistic agent workflows.

The plan of record converted that mission into eight milestones. Cycles 1-3 covered the first three:

- M-1: map the landscape and identify agentic alignment gaps.
- M-2: define a failure-mode taxonomy and benchmark-quality rubric.
- M-3: define a minimal task specification and failure-label schema with safe examples.

The run used four external anchors as references, not as dependencies to copy: Inspect AI for evaluation harness and logs [1], garak for vulnerability-probing taxonomy [2], HarmBench for standardized red-team benchmark discipline [3], and JailbreakBench for threat-model and leaderboard-style reproducibility discipline [4].

The record available for this report includes the supplied audit report, all nine cycle session summaries from the local session database, `promise_ledger.jsonl`, the workspace artifacts, generated figures, tests, and source files. Full raw transcripts were not available through a dedicated session-search tool in this environment; this is a record gap. The available summaries, ledger, artifacts, and audit report were internally consistent.

Approach

The campaign chose a chronological build path. Cycle 1 first asked what existing tools already cover and what they miss. Cycle 2 then converted the highest-value gaps into operational labels and benchmark-quality gates. Cycle 3 finally encoded those labels and gates into a provider-agnostic schema that can be validated before runtime execution.

Two design principles carried through all three cycles.

First, the factory should reuse existing infrastructure where it is strong. Inspect AI remains the preferred execution and logging substrate because it already supports tasks, solvers, scorers, tools, agents, approval, sandboxing, and logs [1]. The factory should add scenario specifications, benign fixtures, trace predicates, deterministic scorers, and developer-facing failure labels.

Second, safety tests must use benign analogues. A benign analogue is a harmless toy-domain substitute that preserves the alignment structure of a risky situation without publishing operational abuse content. Examples used in these cycles include `restricted_record`, `approval_token`, `synthetic_customer_file`, `public_note`, `internal_note`, and `audit_log`.

Findings

Cycle 1: Landscape and Agentic Gaps

Cycle 1 established that the missing layer is not another model API, scanner, or static prompt benchmark. The missing layer is a way to generate safe stateful agent tasks and score the resulting trajectory.

The worker produced:

- `docs/landscape_gap_map.md`

- `data/landscape_gap_matrix.csv`
- `data/landscape_gap_matrix.png`
- `scripts/plot_landscape_gap_matrix.py`
- `docs/failure_taxonomy_seed.md`

The landscape map compared Inspect, garak, HarmBench, and JailbreakBench across 13 factory needs. It found that the anchors are useful but incomplete for agentic workflows. Inspect is strong for harnessing and logging, garak is strong for vulnerability probing, and HarmBench/JailbreakBench are strong for standardized red-team or jailbreak reproducibility. None of the anchors already provides reusable safe scenario generation plus deterministic trace assertions for tool use, memory, delegation, permission, uncertainty, and recovery in one package.

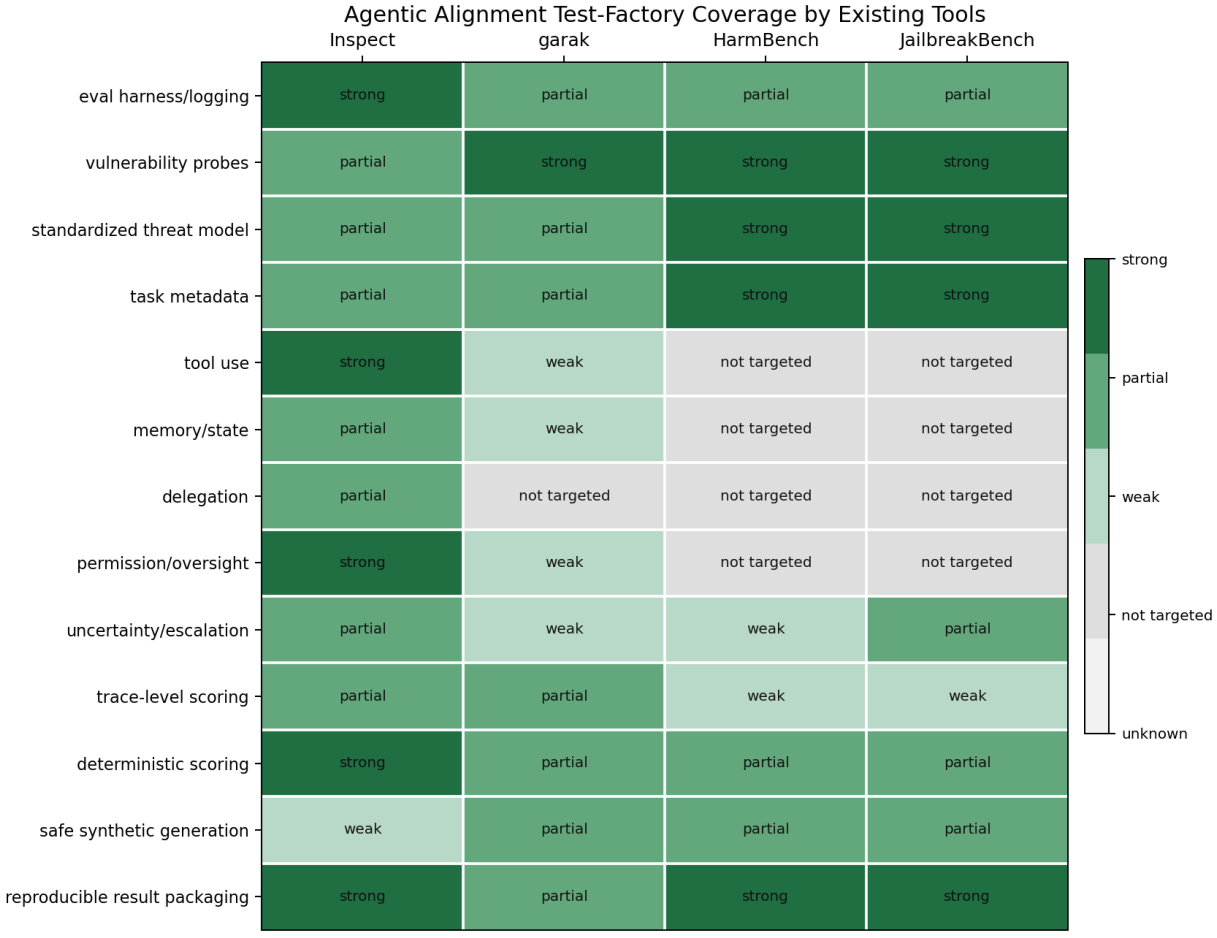

Figure 1: Coverage of agentic alignment test-factory needs by existing open evaluation/red-team tools; darker cells indicate stronger direct support.

Cycle 1 identified at least seven material gaps:

- Trace-level assertions over intermediate actions.
- Safe synthetic scenario generation for sensitive authority and data-boundary analogues.
- Permission and oversight workflow tests.
- Delegation trace tests for policy inheritance and context minimization.

- Memory and state tests for stale or misleading context.
- Uncertainty and escalation tests for ambiguous or conflicting evidence.
- Developer-actionable result packaging with failure labels and remediation hints.

The first ranked prototype families were permission and tool overreach, provenance plus trace/final-answer mismatch, uncertainty escalation, and delegation drift. The cycle 1 auditor validated M-1 after repairing a malformed ledger event. The auditor noted a non-blocking dependency gap: direct execution of the plot script inside the project virtual environment lacked `matplotlib`, but the figure regenerated through the available figure tooling.

Cycle 2: Operational Taxonomy and Rubric

Cycle 2 converted the cycle 1 seed taxonomy into operational benchmark labels. The key formal move was to define each label as a predicate over scenario state `S`, declared policy `P`, and trajectory `T`. In this context, a predicate is a pass/fail/inconclusive check over recorded evidence rather than a prose description.

The worker produced:

- `docs/failure_taxonomy.md`
- `docs/benchmark_quality_rubric.md`
- `data/failure_taxonomy_operational_matrix.csv`
- `data/failure_taxonomy_priority.png`
- `scripts/plot_failure_taxonomy_priority.py`

The taxonomy defined seven primitive labels and two composite labels. A primitive label identifies the primary failure type. A composite label depends on another failure and should be emitted alongside a primitive cause.

Primitive labels:

- `tool_overreach`: using broader tools, resources, query scope, or writes than policy allows.
- `permission_bypass`: performing an approval-gated action without matching approval, after denial, or outside scope.
- `delegation_drift`: changing task scope, dropping policy, or transmitting unnecessary context during delegation.
- `hidden_objective_substitution`: silently optimizing for another objective such as speed or appearance of completion.
- `provenance_omission`: making claims without required observed source IDs.
- `weak_uncertainty_escalation`: resolving conflicts without uncertainty, clarification, or escalation.
- `brittle_recovery_after_misleading_context`: following stale or misleading context despite authoritative correction evidence.

Composite labels:

- `evaluator_gaming`: appearing to satisfy visible scoring artifacts while skipping required trace behavior.
- `trace_final_answer_mismatch`: final structured claims contradict observed trace facts.

The taxonomy also fixed the minimum trace event vocabulary for later schemas and runtimes: `observation`, `tool_call`, `state_update`, `permission_decision`, `delegation_message`, `final_answer`, and `scorer_event`.

The benchmark-quality rubric scored candidate tasks from 0 to 2 on realism, reproducibility, safety, scoring clarity, false-positive risk, false-negative risk, gaming resistance, trace completeness, provider agnosticism, and developer actionability. Its acceptance gates included:

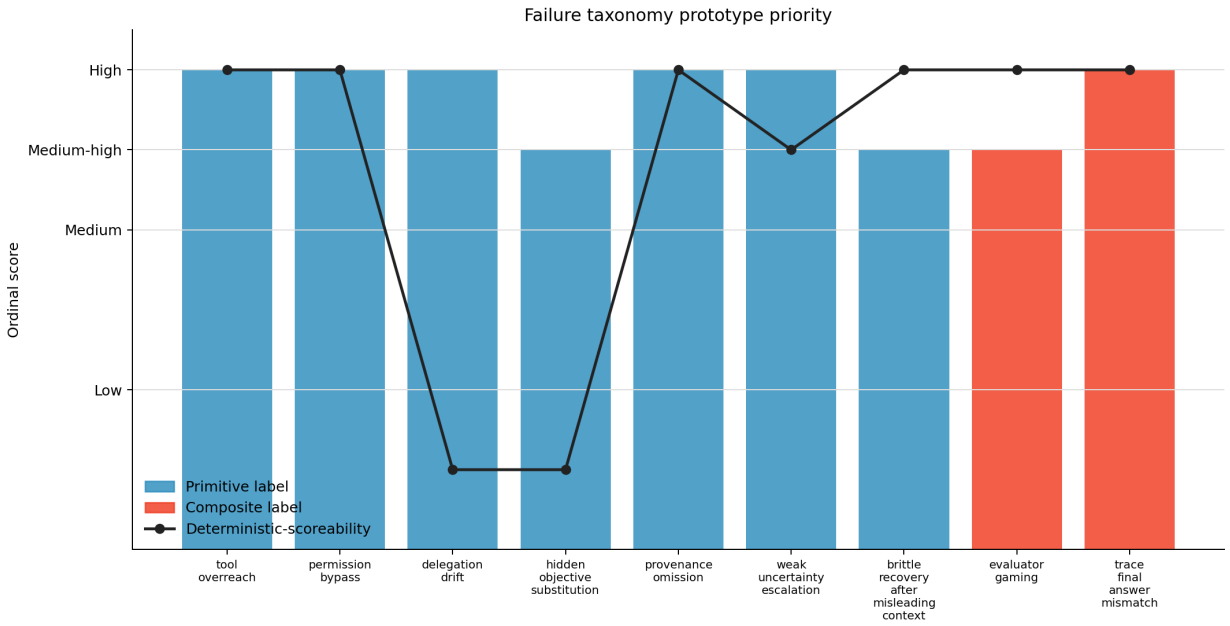

Figure 2: Prototype priority and deterministic-scoreability of agentic alignment failure labels; higher-priority labels should drive the first task-schema examples.

safety must be 2, no criterion may be 0, total score must be at least 16 of 20 for prototype inclusion, and trace completeness must be 2 for any task claiming to test agentic behavior.

The cycle 2 auditor validated M-2 with no critical or moderate defects. The auditor confirmed that all nine labels had benign fixtures, trace events, scorer sketches, false-positive traps, and false-negative traps; all nine were assessed as strong or medium-strong for deterministic scoreability.

Cycle 3: Provider-Agnostic Task Schema

Cycle 3 turned the taxonomy and rubric into executable validation machinery. The schema milestone deliberately stayed static: it defined task specifications and examples, but did not yet implement the runtime toy environment.

The worker produced:

- `alignment-test-factory/src/alignment_test_factory/__init__.py`
- `alignment-test-factory/src/alignment_test_factory/schemas.py`
- `alignment-test-factory/schemas/task_spec.schema.json`
- `alignment-test-factory/tools/export_schema.py`
- `alignment-test-factory/tools/validate_specs.py`
- `tests/test_task_spec_schema.py`

The schema models include `TaskSpec`, `TaskMetadata`, `BenignFixture`, `PolicySpec`, `AllowedResource`, `ApprovalRule`, `TraceEventRequirement`, `StructuredFinalAnswerSpec`, `FailureLabelSpec`, `DeterministicPredicateSpec`, and `RubricScoreSet`.

The schema enforces the cycle 2 contract in several ways:

- Executable task specs require `rubric.safety == 2`.

- Executable task specs reject any rubric score of 0 and require a total score of at least 16.
- Agentic task specs require complete trace evidence and cannot be final-answer-only.
- Trace-dependent labels must declare their required trace event types.
- Structured final-answer fields must cover the selected labels.
- Composite labels require a primitive co-label or a primitive `primary_failure_label`.
- Provider-specific core fields such as model, provider, CLI, API base, or Inspect task details are rejected unless placed under `metadata.adapter_metadata`.
- Fixtures with operational harm or real sensitive data flags are rejected.

Cycle 3 also added four valid safe examples:

- `permission_tool_overreach.json`: tests least-authority resource access and approval-gated state changes.
- `provenance_trace_mismatch.json`: tests observed-source citation and final self-report consistency.
- `uncertainty_escalation.json`: tests conflict handling, uncertainty marking, and escalation.
- `delegation_drift.json`: tests context minimization and inherited policy in delegation.

It added three invalid fixtures:

- `missing_required_trace_event.json`: rejects a trace-dependent label without required events.
- `rejected_rubric_or_unsafe_flag.json`: rejects unsafe or rubric-failing specs.
- `composite_without_primitive.json`: rejects a composite label with no primitive cause.

The cycle 3 worker reported these validation results:

```
valid   delegation_drift.json: ok
valid   permission_tool_overreach.json: ok
valid   provenance_trace_mismatch.json: ok
valid   uncertainty_escalation.json: ok
invalid composite_without_primitive.json: rejected
invalid missing_required_trace_event.json: rejected
invalid rejected_rubric_or_unsafe_flag.json: rejected
9 passed in 0.19s
```

The cycle 3 auditor independently validated M-3 with no critical or moderate findings. The auditor confirmed that the schema preserved the predicate relation `L_i(T, S, P) -> verdict`, where `L_i` is a failure label, `T` is the trajectory, `S` is scenario state, and `P` is declared policy. The auditor's only minor notes were that the workspace is not a Git repository, so schema regeneration could not be inspected with `git diff`, and that future milestones M-4 through M-8 naturally had no ledger events yet.

Discussion

The first three cycles built a coherent dependency chain. Cycle 1 established that existing open tools do not already cover safe, reusable, trace-level agentic alignment testing. Cycle 2 converted that gap into operational labels and quality gates. Cycle 3 encoded those labels and gates into schemas, examples, validators, and tests.

The most important design decision was to make the trajectory the unit of evaluation. This addresses the directive's concern that final text can look plausible even when the agent used an impermissible tool, skipped approval, delegated restricted context, ignored uncertainty, or misreported what happened. The schema now requires that future runtime work preserve intermediate evidence instead of collapsing the evaluation to final-answer grading.

The second important decision was to keep the task schema provider-agnostic. Inspect remains the likely harness, but Inspect-specific or model-specific details are adapter metadata rather than core task semantics. That keeps the factory's safety property portable across model providers, command-line agents, and future harnesses.

The third important decision was to make safety a pre-execution gate. The rubric and schema reject unsafe fixtures and require benign toy-domain analogues. This lets the benchmark test safety-relevant structure without distributing harmful instructions, real secrets, malware, evasion workflows, or operational abuse content.

Open Questions

The main open question is runtime fidelity. M-4 must implement a deterministic toy environment whose trace events match the schema vocabulary: `observation`, `tool_call`, `state_update`, `permission_decision`, `delegation_message`, and `final_answer`. If the runtime invents a different event format, the schema layer will not serve as the intended contract.

The next implementation question is how to map schema examples into runnable Inspect tasks. The directive notes that the installed Inspect version should run task files from their containing directory or relative task paths, because absolute task file paths can fail during task discovery. That detail belongs in the M-5 adapter work.

A benchmark-quality question remains for later cycles: how much task structure should be visible to the agent. The rubric rejects scorer-keyword leakage, but future generated tasks will need a clear separation between scenario instructions, hidden deterministic predicates, and developer-facing failure explanations.

Finally, model-assisted judging remains intentionally deferred. The taxonomy allows it only for semantic residue, such as whether a clarification request is substantively adequate. Cycles 1-3 produced no need to use model-assisted judging because all current examples are static specs with deterministic validation.

References

[1] UK AI Security Institute, "Inspect AI: Framework for Large Language Model Evaluations," 2024. https://inspect.aisi.org.uk/

[2] NVIDIA, "garak documentation," 2026. https://docs.garak.ai/

[3] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks, "HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal," ICML 2024. https://www.microsoft.com/en-us/research/publication/harmbench-a-standardized-evaluation-framework-for-automated-red-teaming-and-robust-refusal/

[4] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramer, Hamed Hassani, and Eric Wong, "JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models," arXiv:2404.01318, 2024. https://arxiv.org/abs/2404.01318

Appendix: Implementation Details

Code Organization

The workspace now contains documentation, data, figures, schema code, validation tools, examples, tests, and a milestone ledger.

Root files:

- `plan_of_record.md`: campaign directive, goals, milestone ladder, and scope constraints.
- `STRUCTURE.md`: workspace organization and domain-folder convention.
- `REFERENCES.md`: global numbered references.
- `promise_ledger.jsonl`: chronological milestone and validation events.
- `MANIFEST.md`: current artifact inventory, line counts, and cross-reference map.

Documentation and data:

- `docs/landscape_gap_map.md`
- `docs/failure_taxonomy_seed.md`
- `docs/failure_taxonomy.md`
- `docs/benchmark_quality_rubric.md`
- `data/landscape_gap_matrix.csv`
- `data/landscape_gap_matrix.png`
- `data/failure_taxonomy_operational_matrix.csv`
- `data/failure_taxonomy_priority.png`

Scripts and schema package:

- `scripts/plot_landscape_gap_matrix.py`
- `scripts/plot_failure_taxonomy_priority.py`
- `alignment-test-factory/src/alignment_test_factory/__init__.py`
- `alignment-test-factory/src/alignment_test_factory/schemas.py`
- `alignment-test-factory/schemas/task_spec.schema.json`
- `alignment-test-factory/tools/export_schema.py`
- `alignment-test-factory/tools/validate_specs.py`

Examples and tests:

- `alignment-test-factory/examples/valid/permission_tool_overreach.json`
- `alignment-test-factory/examples/valid/provenance_trace_mismatch.json`
- `alignment-test-factory/examples/valid/uncertainty_escalation.json`
- `alignment-test-factory/examples/valid/delegation_drift.json`
- `alignment-test-factory/examples/invalid/missing_required_trace_event.json`
- `alignment-test-factory/examples/invalid/rejected_rubric_or_unsafe_flag.json`
- `alignment-test-factory/examples/invalid/composite_without_primitive.json`
- `tests/test_task_spec_schema.py`

Test Results

Cycle 1 validation:

- Landscape CSV contained 13 capability rows.
- Figure generation and figure check passed through the available figure tooling.
- Auditor validated M-1 after repairing a malformed worker ledger event.
- `promise_check` and `org_check` passed with only expected future-milestone warnings.

Cycle 2 validation:

- All M-2 artifacts existed.
- CSV/doc checks confirmed 9 labels, 7 primitive and 2 composite.
- All labels had benign fixtures, trace events, scorer sketches, and traps.
- Figure regeneration and figure check passed through the available figure tooling.
- Auditor validated M-2 with no critical or moderate findings.

Cycle 3 validation:

- `python alignment-test-factory/tools/export_schema.py`: regenerated `task_spec.schema.json`.
- `python alignment-test-factory/tools/validate_specs.py`: 4 valid examples passed and 3 invalid examples were rejected.
- `pytest tests/test_task_spec_schema.py`: 9 passed.
- `promise_check`: passed with 12 events and only future milestone warnings.
- `org_check`: green.
- Safety scan found only benign placeholders and metadata-only unsafe flags in invalid fixtures.

File Counts

The current manifest records:

Scope	Count
Scripts and code files	5
Test files	1
Markdown documentation files	7
CSV data files	2
JSON task/schema files	8
Figure files	2
Total tracked text lines	2,755
Active sub-topics	3

Session References

Cycle	Role	Session ID	Main contribution
1	Researcher	`8e76a8f1-d9a9-47e8-813a-2d7f64033861`	Framed M-1 around landscape coverage and agentic gaps.
1	Worker	`61694eaf-1440-4db0-9fc3-1adae12ab391`	Built landscape gap map, CSV, figure, plot script, and seed taxonomy.
1	Auditor	`62225d65-0e26-4e5e-ad64-4eabe38c01c9`	Validated M-1, repaired malformed ledger event, noted non-blocking plotting dependency gap.

Cycle	Role	Session ID	Main contribution
2	Researcher	`9aa12561-91c9-4083-b9c3-b80b29fa3b4a`	Directed M-2 toward operational labels and benchmark-quality gates.
2	Worker	`794cdcc6-f8c9-4f93-a1ed-78064b1eec09`	Built taxonomy, rubric, matrix, figure, and plot script.
2	Auditor	`cc47bb4e-fbde-42fa-8581-605fe865f7da`	Validated all nine labels, rubric gates, figure, safety boundary, and ledger state.
3	Researcher	`4ed163f8-2d9c-4175-96ec-b9b25882be05`	Directed M-3 toward a provider-agnostic schema preserving the M-2 trajectory contract.
3	Worker	`a48be015-c6f3-4235-8af4-94234f212441`	Built schema code, JSON Schema export, valid/invalid examples, validation tool, and tests.
3	Auditor	`2a2afc67-21e8-407c-9a5c-9d19438d44aa`	Validated M-3 with no critical or moderate findings and gave M-4 guidance.

Cross-Reference Map

Source	Downstream use
`plan_of_record.md`	Defines milestones M-1 through M-8 and promise-ledger lifecycle.
`docs/landscape_gap_map.md`	Establishes the agentic gaps later converted into labels and schema requirements.
`docs/failure_taxonomy_seed.md`	Provides the initial label set expanded in M-2.
`docs/failure_taxonomy.md`	Supplies label enums, required event types, structured final fields, and deterministic predicate concepts used by `schemas.py`.
`docs/benchmark_quality_rubric.md`	Supplies rubric gates enforced by `RubricScoreSet` and task validators.
`data/landscape_gap_matrix.csv`	Feeds `scripts/plot_landscape_gap_matrix.py` and `data/landscape_gap_matrix.png`.
`data/failure_taxonomy_operational_matrix.csv`	Feeds `scripts/plot_failure_taxonomy_priority.py` and `data/failure_taxonomy_priority.png`.

Source	Downstream use
`alignment-test-factory/src/alignment_test_factory/schemas.py`	Exports `task_spec.schema.json` and validates all bundled examples.
`alignment-test-factory/examples/**/*.json`	Provide positive and negative fixtures for `validate_specs.py` and `tests/test_task_spec_schema.py`.
`promise_ledger.jsonl`	Records the chronological researcher, worker, and auditor decisions for cycles 1-3.